

Resolución de código y Análisis Big O

Informe de Lab N°1

Autor: Alvaro Valdebenito
Estructura de Datos y Algoritmos
Fecha: 5 de Abril, 2024

Introducción.

En el siguiente informe, veremos los diferentes tipos de ejercicios propuentos en la Guia de Laboratorio, explicando a su vez el paso a paso. A continuación veremos el desarrollo de sus respectivos ejercicios.

Item 1: Pares entre 2 filas.

A continuación veremos el paso a paso del siguiente código.

```
1  import java.util.*;
2
3  public class Main{
4
5      public static void main(String[] args)
6      {
7          Scanner numerosMatriz = new Scanner(System.
               ↪ in);
8          int N, M, R1, R2;
9
10         System.out.println("Ingrese numero de filas
               ↪ (M) y columnas (N) separadas por un
               ↪ espacio (Estos deben de ser valores
               ↪ entre 1 y 1000):");
11
12         M = numerosMatriz.nextInt();
13         N = numerosMatriz.nextInt();
```

Para la primera parte del código, declaramos las variables, según como lo establecen las condiciones. Se reciben 2 números separados por un espacio, los cuales representan el tamaño de la matriz.

```
1          if(M >= 1 && N >= 1 && M <= 1000 && N <=
               ↪ 1000)
2          {
```

```
3                int[][] matriz = new int[M][N];
4
5                for(int i = 0; i < M; i++)
6                {
7                    System.out.println("Ingrese TODOS
                        ↪ los valores de la Fila [" + (i
                        ↪ + 1) +"] separados por un
                        ↪ espacio:");
8
9                    for(int j = 0; j < N; j++)
10                   {
11
12                       matriz[i][j] = numerosMatriz.
                            ↪ nextInt();
13
14                       while(matriz[i][j] < Math.pow
                            ↪ (10,-8) || matriz[i][j] >
                            ↪ Math.pow(10,8))
15                       {
16                           System.out.println("Ingrese
                                ↪ nuevamente el valor
                                ↪ para la posicion [" + i
                                ↪  +"][" + j + "],
                                ↪ excedi  los limites: "
                                ↪  );
17                           matriz[i][j] = numerosMatriz
                                ↪ .nextInt();
18
19                       }
20                   }
21               }
```

Para esta segunda parte del codigo verificamos que M y N son numeros entre 1 y 1000, si es así, se ingresan los valores de la matriz uno a uno, para luego comprobar que dichos numeros se encuentran entre 10^{-8} y 10^{8}.

```
System.out.println("Ingrese las filas que desea comparar separadas por un espacio: ");
            R1 = numerosMatriz.nextInt();
            R2 = numerosMatriz.nextInt();

            if( (R1 >= 1 && R1 <= M) && (R2 >= 1 && R2 <= M) )
            {
                for(int i=0; i< N ; i++)
                {
                    if(matriz[R1-1][i] % 2==0)
                    {
                        for(int l=0; l< N ; l++)
                        {
                            if(matriz[R2-1][l] % 2==0 && matriz[R2-1][l]==matriz[R1-1][i])
                            {
                                System.out.println("Posicion (" + (R1-1) + "," + l + ") --> " + matriz[R1-1][i] + "/ Posicion (" + (R2-1) + "," + l + ") --> " + matriz[R2-1][l]);
                            }
                        }
                    }
                }
            }

            else
            {
```

```
24                      System.out.println("No se econtr
                           ↪ ningun par en las filas
                           ↪ seleccionadas.");
25                  }
26              }
27
28              else
29              {
30                  System.out.println("La operaci n no fue
                       ↪  realizada, debe ingresar un numero
                       ↪  entre 1 y 1000.");
31              }
32      }
33  }
```

Para la parte final del código, se ingresan las filas que se quieren comparar. Así por último, siguiendo las condiciones del enunciado, se van comparando uno a uno los elementos de cada fila (R1 y R2), de forma que al encontrar un par de números pares , los retorne indicando la posicion en la que se encuentran ambos. Luego finalizan las condiciones para cada if/else, dando termino al código.

Item 2: Ejercicio de Análisis

A continuación veremos los tiempos de ejecución en notación Big O, de los siguientes códigos:

Invertir un número

Primero iremos paso a paso para calcular el Big O de cada función.

```
    import java.util.Scanner;

public class InvertirNumeroDecimal {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Ingresa un numero decimal: ");
        float numero = scanner.nextFloat();

        float numeroInvertido = invertirDecimal(numero);

        System.out.println("Numero invertido: " + numeroInvertido);
    }
```

Para el T(n) de la siguiente función podemos ver que solo hay lineas con valor constante excepto cuando llama a otra función. En este caso se está llamando a la función "invertirDecimal(numero);", que veremos mas adelante.

```
  public static float invertirDecimal(float numero) {
      int parteEntera = (int) numero;
      float parteDecimal = numero - parteEntera;

      int numeroInvertidoEntero = invertirEntero(parteEntera);
      float numeroInvertidoDecimal = invertirDecimalParteDecimal(
 parteDecimal);

      return numeroInvertidoEntero + numeroInvertidoDecimal;
  }
```

Al igual que la función anterior, esta utiliza otra función la cual determinara su grado de complejidad, ya que a simple viste, se puede que a preciar que es O(1), excepto por la funcion a la que se llama, por lo que dependera de esa función.

```java
public static int invertirEntero(int numero) {
    int numeroInvertido = 0;
    while (numero != 0) {
        numeroInvertido = numeroInvertido * 10 + numero % 10;
        numero /= 10;
    }
    return numeroInvertido;
}
```

En este caso, la funcion "InvertirEntero" ya no depende de otra, por lo que si miramos bien, la mayor complejidad que posee la funcion, se encuentra dentro del while, la cual posee un O(n), ya que este es un ciclo que repitira n veces hasta que el numero llegue a 0.

```java
public static float invertirDecimalParteDecimal(float numero) {
    float numeroInvertido = 0;
    float factor = 1;
    while (numero != 0) {
        factor /= 10;
        numeroInvertido += (numero % 10) * factor;
        numero *= 10;
        numero = (int)numero;
    }
    return numeroInvertido;
}
```

Por ultimo, tenemos a la funcion "invertirDecimalParteDecimal" que al igual que la función anterior, esta no llama a ninguna otra, por lo que si la vemos bien, el peor de los casos está dentro del ciclo While, ya que las demás son todas constantes. Dentro del ciclo nos indica que será un O(n), ya que se repitira tantas veces de forma constante hasta que el número sea distinto de 0.
Como conclusión, este codigo posee una complegidad de O(n), independientemente de la función utilizada, ya que ambas funciones que no de penden de las demas, poseen la misma complejidad.

Multiplicación de Matrices

Nuevamente como en el código anterior, iremos separando una a una las funciones, para poder entenderlo de mejor manera.

```
    import java.util.Scanner;

public class MultiplicacionMatrices {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int filasA = scanner.nextInt();
        int columnasA = scanner.nextInt();
        int[][] matrizA = new int[filasA][columnasA];
        ingresarMatriz(scanner, matrizA);

        int filasB = scanner.nextInt();
        int columnasB = scanner.nextInt();
        int[][] matrizB = new int[filasB][columnasB];
        ingresarMatriz(scanner, matrizB);

        if (columnasA != filasB) {
            return;
        }

        int[][] resultado = multiplicarMatrices(matrizA, matrizB);

        imprimirMatriz(resultado);
    }
```

En la primera funcion, que basicamente es el metodo Main, podemos ver como esta depende del resto de funciones al invocarlas, por lo que su Big O dependerá de ellas, ya que si miramos bien el codigo, gran parte tiene constantes las cuales equivalen a O(1) exceptuando a las que llaman a las demas funciones.

```
public static void ingresarMatriz(Scanner scanner, int[][] matriz) {
    for (int i = 0; i < matriz.length; i++) {
        for (int j = 0; j < matriz[0].length; j++) {
            matriz[i][j] = scanner.nextInt();
        }
    }
}
```

Podemos ver que en la funcion "ingresarMatriz", consta de 2 ciclos for, los cuales incrementan de manera constante para ir ingresando los valores de la

matriz. Como son las unicas 2 lineas de ejecución, al calcular el tiempo de ejecución se obtiene O(n^2).

```
 public static int[][] multiplicarMatrices(int[][] matrizA, int[][]
matrizB) {
    int filasA = matrizA.length;
    int columnasA = matrizA[0].length;
    int filasB = matrizB.length;
    int columnasB = matrizB[0].length;

    int[][] resultado = new int[filasA][columnasB];

    for (int i = 0; i < filasA; i++) {
        for (int j = 0; j < columnasB; j++) {
            for (int k = 0; k < columnasA; k++) {
                resultado[i][j] += matrizA[i][k] * matrizB[k][j];
            }
        }
    }

    return resultado;
 }
```

Podemos ver que en la función "multiplicarMatrices" se declaran las 5 primeras lineas como constantes, es decir O(1), en esta función, podemos ver que el tiempo de ejecución va a estar dado por los ciclos anidados, que en este caso serian 3 ciclos for, y a su vez incrementan de manera constante, por lo que el Big O de esta función esta dado por O(n^3).

```
    public static void imprimirMatriz(int[][] matriz) {
        for (int[] fila : matriz) {
            for (int elemento : fila) {
                System.out.print(elemento + " ");
            }
            System.out.println();
        }
    }
}
```

Por ultimo, en la función "imprimirMatriz", esta posee 2 ciclos anidados, siendo estos los de mayor complejidad, por lo que el ciclo estaria dado por un tiempo de ejecución de O(n^2).

Se concluye que el metodo main, al estar compuestas por otras funciones, la que tendria mayor tiempo de ejecución sería la función "multiplicarMatrices", lo que a su vez provoca que el metodo main posea su misma complejidad, siendo esta O(n^3).